

Operating Systems · Course Syllabus

Course Information

- **Subject**: Operating Systems
- **Duration**: 1 Semester
- **Credits**: 4

Unit 1: Introduction to Operating Systems

1.1 What is an Operating System?

An Operating System (OS) is system software that manages computer hardware, software resources, and provides common services.

Functions of an OS:

- Process management: creating, scheduling, and terminating processes
- Memory management: allocation and deallocation of memory to processes
- File system management: creating, deleting, reading, and writing files
- I/O device management: managing input/output devices and their drivers
- Security and protection: controlling access to system resources

Types of Operating Systems:

- Batch Operating System: executes jobs in batches without user interaction
- Time-Sharing OS: allows multiple users to share system resources simultaneously
- Distributed OS: manages a group of independent computers as a single system
- Real-Time OS (RTOS): processes data within a guaranteed time constraint
- Embedded OS: designed for embedded computer systems (IoT devices, routers)

1.2 System Calls

System calls provide the interface between a process and the operating system. They are the only way user programs can request services from the OS.

Categories:

- Process control: `fork()`, `exec()`, `exit()`, `wait()`
- File management: `open()`, `read()`, `write()`, `close()`
- Device management: `ioctl()`, `read()`, `write()`
- Information maintenance: `getpid()`, `alarm()`, `sleep()`
- Communication: `pipe()`, `shmget()`, `mmap()`

Unit 2: Process Management

2.1 Process Concepts

A process is a program in execution. It includes the program code, current activity (program counter), stack, data section, and heap.

Process States:

- New: process is being created
- Ready: process is waiting to be assigned to a processor
- Running: instructions are being executed
- Waiting/Blocked: process is waiting for some event (I/O completion)
- Terminated: process has finished execution

Process Control Block (PCB): Contains process state, program counter, CPU registers, scheduling information, memory management information, etc.

2.2 Process Scheduling

The process scheduler selects which process runs on the CPU at any given time.

Scheduling Criteria:

- CPU utilization: keep the CPU as busy as possible
- Throughput: number of processes completed per unit time
- Turnaround time: total time from submission to completion
- Waiting time: total time spent in the ready queue
- Response time: time from submission to first response

2.3 CPU Scheduling Algorithms

First-Come, First-Served (FCFS):

- Simplest scheduling algorithm
- Non-preemptive: process runs until completion
- Convoy effect: short processes wait behind long ones
- Average waiting time can be high

Shortest Job First (SJF):

- Selects process with smallest burst time
- Optimal average waiting time
- Difficult to know burst time in advance
- Can be preemptive (SRTF) or non-preemptive

Round Robin (RR):

- Each process gets a fixed time quantum (10-100ms)
- Preemptive: process is moved to back of queue after quantum expires
- Good for time-sharing systems
- Performance depends on quantum size: too small = too much context switching, too large = becomes FCFS

Priority Scheduling:

- Each process assigned a priority number
- CPU allocated to highest priority process
- Can lead to starvation: low priority processes may never execute
- Solution: aging · gradually increase priority of waiting processes

Unit 3: Memory Management

3.1 Memory Hierarchy

Memory is organized in a hierarchy: registers (fastest, smallest) -> cache -> main memory (RAM) -> secondary storage (disk).

3.2 Contiguous Memory Allocation

Each process is allocated a single contiguous block of memory.

****Fixed Partitioning:**** Memory divided into fixed-size partitions. Internal fragmentation occurs when a process is smaller than its partition.

****Variable Partitioning:**** Partitions created dynamically based on process size. External fragmentation occurs when free memory is scattered in small blocks.

Allocation Strategies:

- First Fit: allocate the first block that is big enough
- Best Fit: allocate the smallest block that is big enough
- Worst Fit: allocate the largest available block

3.3 Paging

Paging divides physical memory into fixed-size frames and logical memory into pages of the same size. A page table maps logical pages to physical frames.

Advantages:

- No external fragmentation
- Allows non-contiguous memory allocation

- Enables sharing of memory between processes

****Page Table:**** Each process has its own page table. Translation Lookaside Buffer (TLB) is a cache for page table entries, speed

3.4 Virtual Memory

Virtual memory allows a process to use more memory than physically available. Only needed pages are loaded into memory.

****Demand Paging:**** Pages are loaded only when referenced (lazy loading). If a page is not in memory, a page fault occurs, and th

Page Replacement Algorithms:

- FIFO: replace the oldest page · simple but Belady's anomaly possible
- LRU (Least Recently Used): replace the page not used for the longest time · good performance, expensive to implement
- Optimal: replace the page that will not be used for the longest future time · theoretical best, impossible to implement in practice

****Thrashing:**** When a process spends more time paging than executing. Occurs when the working set exceeds available memory

Unit 4: Deadlocks

4.1 Deadlock Conditions

A deadlock occurs when two or more processes are waiting for each other to release resources.

Four necessary conditions (all must hold simultaneously):

1. Mutual Exclusion: a resource can be held by only one process at a time
2. Hold and Wait: a process holding resources can request additional resources
3. No Preemption: resources cannot be forcibly taken from a process
4. Circular Wait: a circular chain of processes, each waiting for a resource held by the next

4.2 Deadlock Handling Strategies

****Prevention:**** Eliminate one of the four conditions. For example, impose ordering on resource requests to prevent circular wait.

****Avoidance:**** Use algorithms like the Banker's Algorithm to ensure the system never enters an unsafe state.

****Banker's Algorithm:**** Checks if granting a resource request would leave the system in a safe state (where all processes can finish)

****Detection and Recovery:**** Allow deadlocks to occur, detect them using a resource allocation graph, then recover by killing processes

Unit 5: File Systems

5.1 File Concepts

A file is a named collection of related information stored on secondary storage.

****File Attributes:**** name, identifier, type, location, size, protection, timestamps.

****File Operations:**** create, read, write, delete, seek, truncate, append.

5.2 Directory Structure

- Single-level directory: all files in one directory · naming collisions
- Two-level directory: separate directory for each user
- Tree-structured directory: hierarchical, subdirectories allowed
- Acyclic graph directory: allows shared files/directories (links)

5.3 File Allocation Methods

****Contiguous Allocation:**** Each file stored in contiguous disk blocks. Fast sequential access but suffers from external fragmentation

****Linked Allocation:**** Each block contains a pointer to the next block. No fragmentation but slow random access. FAT (File Allocation Table)

****Indexed Allocation:**** An index block contains pointers to all file blocks. Fast random access, no fragmentation, but index block o

5.4 Free Space Management

- Bit vector/bitmap: one bit per block (0 = free, 1 = allocated)
- Linked list: free blocks linked together
- Grouping: first free block stores addresses of n free blocks
- Counting: keep track of contiguous free blocks with (start, count) pairs

Unit 6: Disk Scheduling

6.1 Disk Scheduling Algorithms

****FCFS (First-Come, First-Served):**** Service requests in arrival order. Simple but high seek time.

****SSTF (Shortest Seek Time First):**** Service the request closest to current head position. Better average seek time but can cause

****SCAN (Elevator Algorithm):**** Head moves in one direction servicing requests, then reverses. No starvation. Also called the eleva

****C-SCAN (Circular SCAN):**** Head moves in one direction only, then jumps back to the beginning. Provides more uniform wait tim

****LOOK and C-LOOK:**** Variations of SCAN/C-SCAN where the head only goes as far as the last request in each direction, rather